



Why Bandits Matter

Sequential Decision-Making Beyond Supervised Learning

Author: Tang

Institute: Tang's Machine Learning Blog

Date: 2026-06-15

Version: 1.0.0

Topic: Bandits, Online Learning, Sequential Decision-Making

Contents

Before the First Formula	1
Chapter 1 The First Object Is the Feedback	2
1.1 Do not start with a tuple	2
1.2 The missing column	2
1.3 Why bad actions damage future data	3
Chapter 2 A Small Experiment Before More Theory	4
2.1 The same world, two ways to learn	4
2.2 The algorithms in code	4
2.3 What the code produces	6
2.4 The lesson hidden in the experiment	8
Chapter 3 Probability, Introduced Only When Needed	9
3.1 A reward is a number we have not seen yet	9
3.2 The sample average is a noisy ruler	9
3.3 Turning a tail bound into an error bar	10
3.4 The union bound is not mysterious	11
3.5 History means what the learner already knows	11
Chapter 4 Regret Is the Price of Learning	12
4.1 The meaning before the formula	12
4.2 Now the notation earns its place	12
4.3 The key decomposition	12
4.4 Checking the switch by hand	13
4.5 Taking expectation	13
Chapter 5 A Simple Failure: Explore-Then-Commit	15
5.1 Taste first, then order forever	15
5.2 The two costs	15
5.3 Bounding the wrong commitment probability	15
5.4 What this simple algorithm teaches	16
Chapter 6 Optimism: Trust the Unknown Until It Disappoints You	18
6.1 The idea in one sentence	18
6.2 Where the radius comes from	18
6.3 The good event	19
6.4 Why a bad arm eventually loses	20
6.5 The regret bound	21
Chapter 7 Sampling a Possible World	22
7.1 The Bayesian picture	22

7.2	A coin-click model	22
7.3	The decision rule	23
7.4	Why this explores	23
7.5	The identity slowly	23
7.6	Information accounting	24
Chapter 8 Why This Toy Model Keeps Reappearing		25
8.1	Context changes the arms	25
8.2	Linear bandits: the geometry version	25
8.3	Gaussian-process bandits: smooth unknown landscapes	26
8.4	Reliable AI and the same old missing column	26
Epilogue: The Essence		27
References		28

Before the First Formula

The first question in a learning problem is not “which model should we use?” It is more primitive:

Where did the data come from?

In supervised learning, the data usually arrive as a table. Someone has already collected the inputs and labels. The learner may be clever or foolish, but the table is there.

In bandit learning, the table is not there. The learner helps create it. Each action asks the world one question, and the world answers only that question.

This is the whole difference. A bandit algorithm is not merely predicting from data. It is choosing which data it will get to see.

The shortest useful definition

A bandit problem is learning by trying one option at a time, while never seeing what would have happened under the options not tried.

That sentence is enough to begin. The mathematics enters only because we want to measure how costly this kind of learning is, and how a good algorithm keeps the cost under control.

A helpful mental picture is an ordinary recommendation slot. At 8:00 a.m., the system chooses one item to show. A user clicks or does not click. The system sees the outcome of the shown item, but it does not see the click outcomes of the hidden items. Tomorrow it must choose again, using yesterday’s incomplete evidence.

This example is not special. The same shape appears when a teacher chooses one hint for a student, a doctor chooses one treatment, a lab chooses one experiment, or a prompt-selection system chooses one prompt for a language model. In all cases, the learner receives feedback from the chosen action, not from the unchosen alternatives.

The goal of this note is to keep that simple picture visible while building the main formulas. The style follows a useful tradition in modern bandit writing: start from the feedback pattern, use examples to make the pressure of the problem visible, and introduce proof machinery only when it pays rent. Lattimore and Szepesvari describe their book as mathematically focused but not a traditional chain of lemmas and theorems; they emphasize guiding principles, intuition, and depth over formal display [4]. That is the standard I have in mind here.

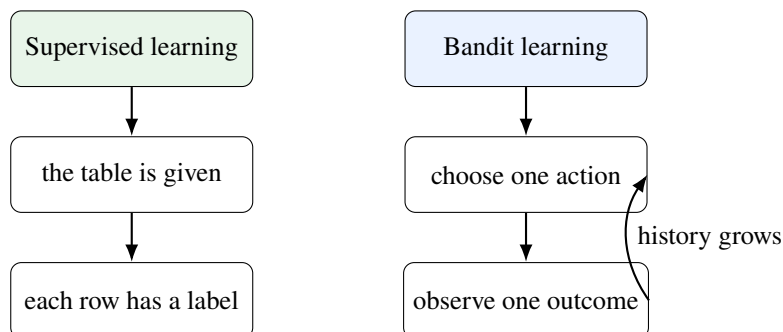


Figure 1: Supervised learning starts from a table. Bandit learning builds a table by acting.

Chapter 1 The First Object Is the Feedback

1.1 Do not start with a tuple

A common weak opening is to define a bandit problem as a list of symbols. That is technically correct and pedagogically backwards. The first object is not a tuple. The first object is the feedback.

Suppose there are several actions. At a given moment, you choose one. The world returns one number. The other numbers remain unobserved.

Only now do the symbols become useful. Let

$$A_t$$

be the action chosen at time t , and let

$$X_t(A_t)$$

be the reward observed after choosing it. If $a \neq A_t$, then $X_t(a)$ is not observed.

The feedback pattern

$$\begin{array}{ll} \text{choose } A_t & \implies \text{observe } X_t(A_t), \\ \text{do not choose } a & \implies \text{do not observe } X_t(a). \end{array}$$

The second line is the source of almost everything in bandit theory. The missing outcome is not a nuisance. It is the problem.

1.2 The missing column

Imagine two possible actions, A and B . For a single user, there are two potential outcomes:

$$X(A), \quad X(B).$$

One says what would happen if we chose A . The other says what would happen if we chose B . We can reveal only one.

chosen action	$X(A)$	$X(B)$	observed outcome
A	visible	hidden	$X(A)$
B	hidden	visible	$X(B)$

A supervised-learning dataset tries to fill the label column for every example. A bandit dataset has a hole in every row. The hole is exactly where the unchosen actions would have been.

A useful plain-language test

If after acting you can ask, “what would have happened if I had chosen differently?” and the data cannot answer, you are in bandit territory.

1.3 Why bad actions damage future data

In supervised learning, a bad prediction is painful, but the label still arrives. In bandit learning, a bad action can also damage the learner's future evidence.

If an algorithm always chooses the currently popular item, it keeps collecting data about that item. The less popular item may remain uncertain forever. The learner is not only using biased data; it is generating the bias.

This is why bandit algorithms need exploration. Exploration is not curiosity for its own sake. It is the repair mechanism for missing feedback.

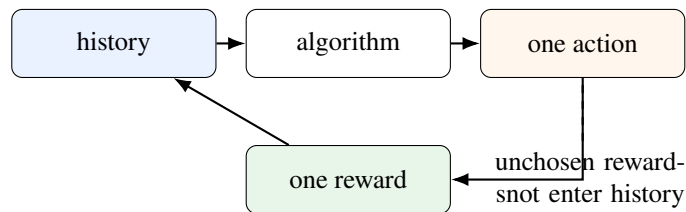


Figure 1.1: The learner's action controls the next piece of data. This feedback loop is the main difference from ordinary supervised learning.

Chapter 2 A Small Experiment Before More Theory

2.1 The same world, two ways to learn

Before adding more probability, it is useful to run a tiny experiment. The point is not to make a benchmark. The point is to make the feedback pattern visible.

There are three arms. You may think of them as three possible thumbnails, three prompts, or three treatments in a toy clinical example. Their click probabilities are

$$\mu_1 = 0.10, \quad \mu_2 = 0.12, \quad \mu_3 = 0.16.$$

Arm 3 is best, but the learner does not know this.

We compare four learners in exactly the same world.

- **Full-feedback learner.** It chooses one arm but observes the reward of every arm. This is the supervised-learning spirit: labels are supplied by the data-collection process rather than purchased one action at a time.
- **Greedy bandit.** It tries each arm once, then always chooses the arm with the largest observed average.
- **UCB.** It chooses the arm with the largest empirical mean plus an uncertainty bonus.
- **Thompson sampling.** It samples a plausible click rate for every arm from its posterior and chooses the arm whose sampled value is largest.

The full-feedback learner is not meant to be a general supervised-learning algorithm. It is the cleanest possible full-information analogue. Its purpose is to isolate the difference between seeing all labels and seeing only the label of the action we picked.

What the experiment should reveal

If every round gives labels for all arms, learning is mostly estimation. If every round gives only one label, learning is also data acquisition. The learner must decide which uncertainty is worth reducing.

2.2 The algorithms in code

The code below is deliberately plain. No library hides the learning rule. Each algorithm keeps a few counters and chooses an arm.

A minimal reproducible simulation for full feedback, greedy bandits, UCB, and Thompson sampling.

```
1 import numpy as np
2
3 def run_once(p, T, rng):
4     K = len(p)
5     best = np.max(p)
6     regret = {"full": [], "greedy": [], "ucb": [], "ts": []}
7
8     # 1. Full feedback: choose one arm, observe all arms.
9     sums = np.zeros(K)
10    counts = np.zeros(K)
```

```

11  for t in range(T):
12      if t < K:
13          a = t
14      else:
15          a = np.argmax(sums / np.maximum(counts, 1))
16          regret["full"].append(best - p[a])
17
18      rewards_all = rng.binomial(1, p)  # the full row is visible
19      sums += rewards_all
20      counts += 1
21
22  # 2. Greedy bandit: observe only the chosen arm.
23  sums = np.zeros(K)
24  counts = np.zeros(K)
25  for t in range(T):
26      if t < K:
27          a = t
28      else:
29          a = np.argmax(sums / np.maximum(counts, 1))
30          r = rng.binomial(1, p[a])  # only this reward is visible
31          regret["greedy"].append(best - p[a])
32          sums[a] += r
33          counts[a] += 1
34
35  # 3. UCB: empirical mean plus an uncertainty bonus.
36  sums = np.zeros(K)
37  counts = np.zeros(K)
38  for t in range(T):
39      if t < K:
40          a = t
41      else:
42          mean = sums / counts
43          radius = np.sqrt(2 * np.log(t + 1) / counts)
44          a = np.argmax(mean + radius)
45          r = rng.binomial(1, p[a])
46          regret["ucb"].append(best - p[a])
47          sums[a] += r
48          counts[a] += 1
49
50  # 4. Thompson sampling for Bernoulli rewards.
51  alpha = np.ones(K)
52  beta = np.ones(K)
53  for t in range(T):
54      sample = rng.beta(alpha, beta)
55      a = np.argmax(sample)
56      r = rng.binomial(1, p[a])
57      regret["ts"].append(best - p[a])

```



```

58     alpha[a] += r
59     beta[a] += 1 - r
60
61     return {name: np.cumsum(vals) for name, vals in regret.items()}

```

Read the code slowly. The full-feedback learner has the line

```
rewards_all = rng.binomial(1, p).
```

That line gives it a whole row of labels. The bandit algorithms have the line

```
r = rng.binomial(1, p[a]).
```

That line gives them only the chosen label. The difference between these two lines is the difference between supervised-style feedback and bandit feedback.

2.3 What the code produces

I ran the experiment for $T = 2000$ rounds and averaged over 400 independent runs. The table reports the final cumulative regret and the average number of pulls of each arm.

Algorithm	Final regret	Pulls of arm 1	Pulls of arm 2	Pulls of arm 3
Full feedback	4.57	26.2	74.8	1899.0
Greedy bandit	98.08	1504.0	196.0	300.0
UCB	49.81	448.7	572.2	979.1
Thompson sampling	21.23	160.6	289.8	1549.5

Table 2.1: Average behavior over 400 runs with true means (0.10, 0.12, 0.16) and horizon $T = 2000$.

The numbers tell the story without any theorem.

The full-feedback learner rapidly finds the best arm because every round teaches it about every arm. Greedy often gets trapped by early noise. UCB keeps checking uncertain arms because uncertainty itself becomes part of the index. Thompson sampling explores in a softer way: an arm is tried when its posterior still gives it a plausible chance of being best.

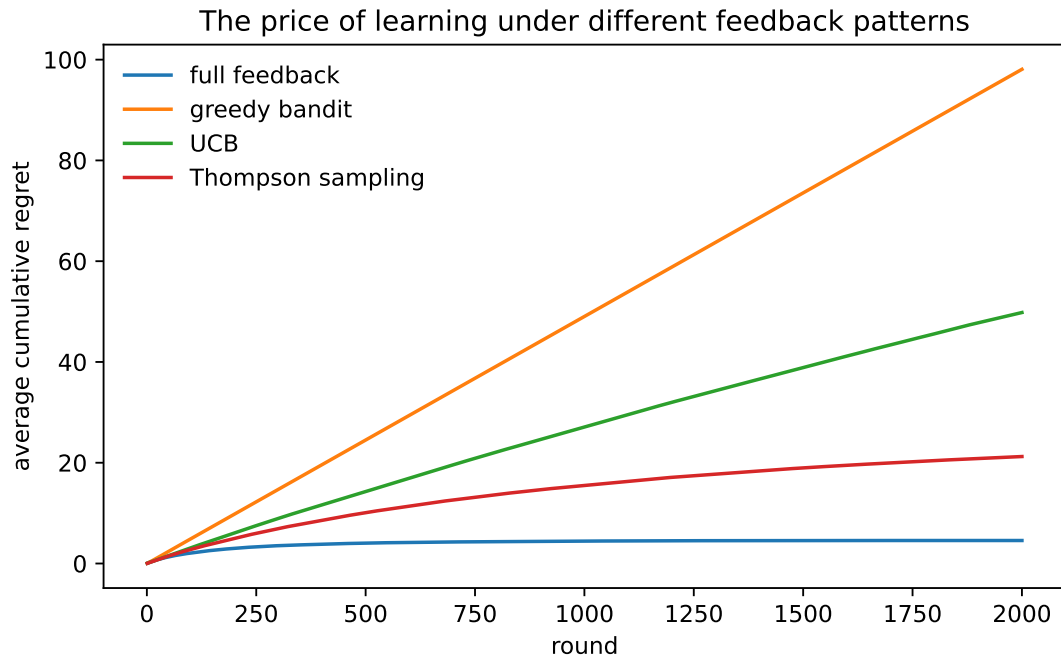


Figure 2.1: Average cumulative regret. The greedy bandit can pay a large price for believing early noise. UCB and Thompson sampling reduce this cost by making uncertainty operational.

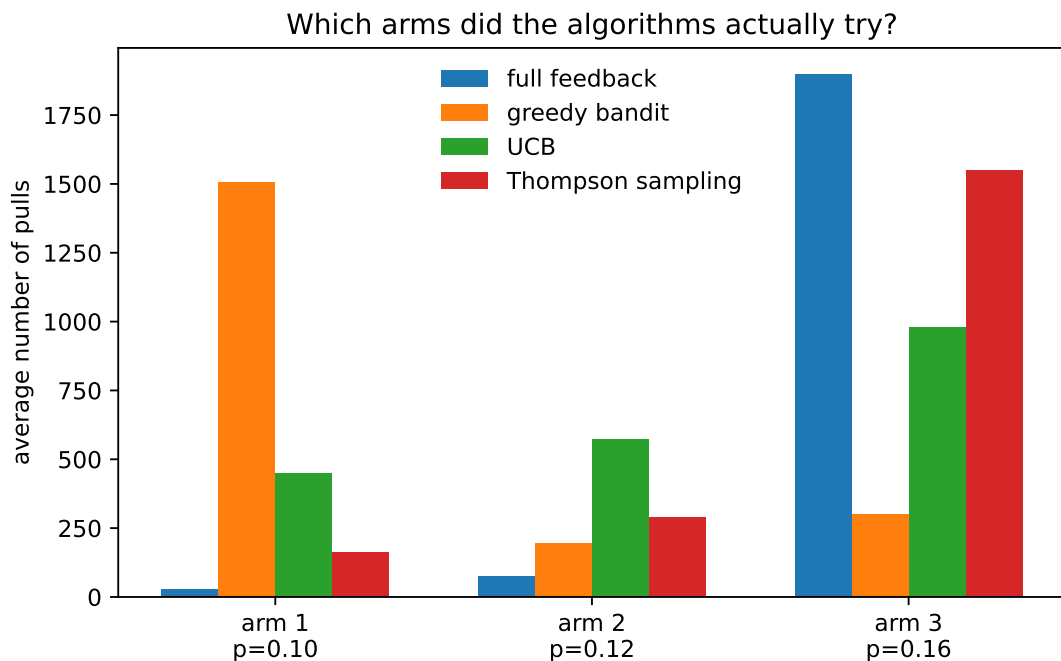


Figure 2.2: Average pull counts. The best arm is arm 3. The greedy learner often spends many rounds on the wrong arm; the sampling and optimism-based methods move probability mass toward the best arm while still collecting evidence.

2.4 The lesson hidden in the experiment

A supervised learner can be viewed as a reader of a completed table. A bandit learner is a writer of an incomplete table. This makes a qualitative difference.

The one-line difference

full feedback:	$(X_t(1), X_t(2), X_t(3))$ is observed,
bandit feedback:	$X_t(A_t)$ is observed, and $X_t(a)$ for $a \neq A_t$ is missing.

Now the later formulas have a concrete object to explain. Regret measures the cost of not knowing the best arm. Confidence intervals explain why UCB does not blindly trust early averages. Posterior sampling explains why Bayesian uncertainty can be converted into action.

Chapter 3 Probability, Introduced Only When Needed

3.1 A reward is a number we have not seen yet

Probability can sound abstract when introduced as a formal system. In bandits, its first role is modest: it gives language for numbers we have not seen yet.

Before an item is shown, its click outcome is unknown. After it is shown, the outcome becomes a number. We model the before-state by a random variable.

For a click/no-click reward,

$$X = \begin{cases} 1, & \text{click,} \\ 0, & \text{no click.} \end{cases}$$

If the click probability is μ , then

$$\mathbb{P}(X = 1) = \mu, \quad \mathbb{P}(X = 0) = 1 - \mu.$$

The average value of this random reward is

$$\begin{aligned} \mathbb{E}[X] &= 1 \cdot \mathbb{P}(X = 1) + 0 \cdot \mathbb{P}(X = 0) \\ &= 1 \cdot \mu + 0 \cdot (1 - \mu) \\ &= \mu. \end{aligned}$$

So in a click model, the mean reward is simply the click probability.

3.2 The sample average is a noisy ruler

If we try the same arm n times, we see rewards

$$Y_1, Y_2, \dots, Y_n.$$

The empirical mean is

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n Y_i.$$

This is the observed click rate. It is our ruler for the unknown mean μ .

But the ruler is noisy. If a thumbnail has true click probability 0.10, it can still receive 2 clicks in 10 trials or 0 clicks in 10 trials. Small samples wobble.

The first probability question is therefore:

How far can the observed average be from the true average?

For bounded rewards in $[0, 1]$, Hoeffding's inequality says that large errors become exponentially unlikely:

$$\mathbb{P}(|\hat{\mu}_n - \mu| \geq r) \leq 2 \exp(-2nr^2).$$

This line is not meant to be memorized as a magic spell. Read it from right to left:

$$\begin{aligned} n \text{ grows} &\implies \exp(-2nr^2) \text{ shrinks,} \\ r \text{ grows} &\implies \exp(-2nr^2) \text{ shrinks.} \end{aligned}$$

More data makes a fixed error less likely. A larger error is also less likely.

3.3 Turning a tail bound into an error bar

Algorithms usually want an error bar. That means choosing r so that the probability of error is at most a small number δ :

$$\mathbb{P}(|\hat{\mu}_n - \mu| \geq r) \leq \delta.$$

Hoeffding gives the sufficient condition

$$2 \exp(-2nr^2) = \delta.$$

Solve this slowly:

From Hoeffding to a confidence radius

$$\begin{aligned} 2 \exp(-2nr^2) &= \delta \\ \exp(-2nr^2) &= \frac{\delta}{2} \\ \log(\exp(-2nr^2)) &= \log\left(\frac{\delta}{2}\right) \\ -2nr^2 &= \log\left(\frac{\delta}{2}\right) \\ 2nr^2 &= \log\left(\frac{2}{\delta}\right) \\ r^2 &= \frac{\log(2/\delta)}{2n} \\ r &= \sqrt{\frac{\log(2/\delta)}{2n}}. \end{aligned}$$

Thus a clean error bar is

$$|\hat{\mu}_n - \mu| \leq \sqrt{\frac{\log(2/\delta)}{2n}}$$

with probability at least $1 - \delta$.

The idea behind the formula

An arm tried only a few times deserves a wide error bar. An arm tried many times deserves a narrow one. UCB is built from exactly this sentence.

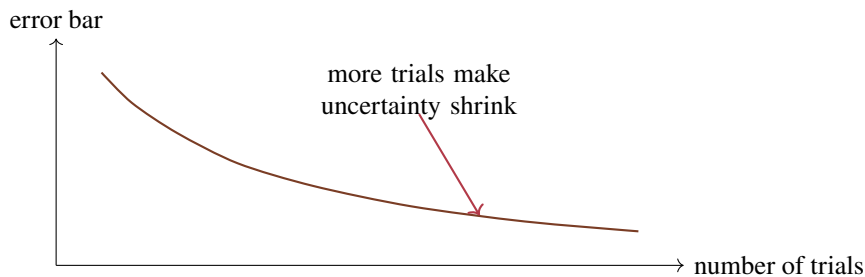


Figure 3.1: The confidence radius shrinks like $1/\sqrt{n}$. This shrinking error bar is the engine inside UCB.

3.4 The union bound is not mysterious

Many proofs need a statement to hold for many arms and many times. We first control one error event, then combine many such controls.

The only tool needed is the union bound:

$$\mathbb{P}(E_1 \cup E_2 \cup \dots \cup E_m) \leq \mathbb{P}(E_1) + \mathbb{P}(E_2) + \dots + \mathbb{P}(E_m).$$

Why is this true? If two alarms can ring, the chance that at least one rings cannot exceed the sum of their separate chances. The sum may double-count worlds where both alarms ring, so it is an upper bound.

For two events, the exact identity is

$$\mathbb{P}(E_1 \cup E_2) = \mathbb{P}(E_1) + \mathbb{P}(E_2) - \mathbb{P}(E_1 \cap E_2).$$

Since probabilities are nonnegative,

$$\mathbb{P}(E_1 \cap E_2) \geq 0,$$

so

$$\mathbb{P}(E_1 \cup E_2) \leq \mathbb{P}(E_1) + \mathbb{P}(E_2).$$

For many events, repeat the same idea.

3.5 History means what the learner already knows

Bandit data are adaptive: today's action depends on yesterday's observations. To talk about this without mysticism, we use the word history.

Let $\mathcal{F}_t$ denote everything known before choosing at time t :

$$\mathcal{F}_t = \{A_1, X_1(A_1), \dots, A_{t-1}, X_{t-1}(A_{t-1})\}.$$

An algorithm chooses A_t using this history. Then the new reward arrives and the history grows.

Conditional probability simply means probability after opening the notebook:

$$\mathbb{P}(\text{event} \mid \mathcal{F}_t) = \text{probability of the event given the current notebook.}$$

A nontechnical reading of $\mathcal{F}_t$

The symbol $\mathcal{F}_t$ is not there to impress the reader. It means the notebook of everything the learner has seen so far.

This is enough probability for the first bandit algorithms: random rewards, means, sample averages, error bars, union bounds, and history.

Chapter 4 Regret Is the Price of Learning

4.1 The meaning before the formula

A bandit learner must sometimes choose an action that may not be best, because otherwise it may never learn what is best. The cost of these imperfect choices is called regret.

Think of regret as the price paid for not already knowing the best action. If the best thumbnail would have produced a 10% click rate and the chosen thumbnail has an 8% click rate, the expected loss for that decision is 2% of a click.

This does not mean exploration is bad. Exploration is often the only way to stop making the same comfortable mistake. Regret is not a moral judgment. It is accounting.

4.2 Now the notation earns its place

Suppose there are K arms. Arm a has mean reward

$$\mu_a = \mathbb{E}[X(a)].$$

The best mean is

$$\mu_* = \max_{1 \leq a \leq K} \mu_a.$$

If the learner chooses A_t , then the expected loss at that time is

$$\mu_* - \mu_{A_t}.$$

Adding over T rounds gives pseudo-regret:

$$\text{Reg}_T = \sum_{t=1}^T (\mu_* - \mu_{A_t}).$$

This compares the learner with the best fixed arm, not with a magical oracle that knows every random outcome in advance.

4.3 The key decomposition

For each arm define its gap

$$\Delta_a = \mu_* - \mu_a,$$

and define the number of times arm a is played by

$$N_a(T) = \sum_{t=1}^T \mathbf{1}\{A_t = a\}.$$

The indicator $\mathbf{1}\{A_t = a\}$ is just a switch:

$$\mathbf{1}\{A_t = a\} = \begin{cases} 1, & A_t = a, \\ 0, & A_t \neq a. \end{cases}$$

Now decompose regret:

Regret as gap times count

$$\begin{aligned}
\text{Reg}_T &= \sum_{t=1}^T (\mu_* - \mu_{A_t}) \\
&= \sum_{t=1}^T \sum_{a=1}^K (\mu_* - \mu_a) \mathbf{1}\{A_t = a\} \\
&= \sum_{a=1}^K (\mu_* - \mu_a) \sum_{t=1}^T \mathbf{1}\{A_t = a\} \\
&= \sum_{a=1}^K \Delta_a N_a(T).
\end{aligned}$$

This formula is the first serious piece of bandit analysis. It says that a bad arm hurts in two ways: how bad it is, and how often it is played.

4.4 Checking the switch by hand

Suppose that at time t the algorithm chose arm j . Then exactly one indicator is equal to 1:

$$\mathbf{1}\{A_t = j\} = 1, \quad \mathbf{1}\{A_t = a\} = 0 \quad (a \neq j).$$

Therefore

$$\begin{aligned}
\sum_{a=1}^K (\mu_* - \mu_a) \mathbf{1}\{A_t = a\} &= (\mu_* - \mu_1) \mathbf{1}\{j = 1\} + \cdots + (\mu_* - \mu_K) \mathbf{1}\{j = K\} \\
&= 0 + \cdots + 0 + (\mu_* - \mu_j) + 0 + \cdots + 0 \\
&= \mu_* - \mu_j \\
&= \mu_* - \mu_{A_t}.
\end{aligned}$$

No probability is being used here. It is bookkeeping.

4.5 Taking expectation

The count $N_a(T)$ is random because the algorithm's choices depend on random rewards. Its expectation is easy to interpret.

For any event E ,

$$\mathbb{E}[\mathbf{1}\{E\}] = 1 \cdot \mathbb{P}(E) + 0 \cdot \mathbb{P}(E^c) = \mathbb{P}(E).$$

Therefore

$$\begin{aligned}
\mathbb{E}[N_a(T)] &= \mathbb{E} \left[\sum_{t=1}^T \mathbf{1}\{A_t = a\} \right] \\
&= \sum_{t=1}^T \mathbb{E}[\mathbf{1}\{A_t = a\}] \\
&= \sum_{t=1}^T \mathbb{P}(A_t = a).
\end{aligned}$$

Taking expectation in the regret decomposition gives

$$\mathbb{E}[\text{Reg}_T] = \sum_{a=1}^K \Delta_a \mathbb{E}[N_a(T)].$$

Takeaway

To prove a regret bound, we usually prove that bad arms are not played too often. Everything else is a way of making that sentence true.

Chapter 5 A Simple Failure: Explore-Then-Commit

5.1 Taste first, then order forever

A natural first strategy is simple. Try each thumbnail for a while. Estimate its click rate. Then use the one that looks better.

This is called explore-then-commit. The name is plain because the algorithm is plain.

A restaurant analogy

A friend says: try two restaurants twice, then pick one for every dinner this month. This may sound reasonable. But if two restaurants are close in quality, two visits are not enough. If one restaurant is clearly bad, too many test visits are wasteful. The fixed trial length is both too small and too large.

Let there be two arms. Arm 1 is better:

$$\mu_1 > \mu_2, \quad \Delta = \mu_1 - \mu_2 > 0.$$

Explore-then-commit pulls each arm m times. Then it chooses the arm with larger empirical mean.

5.2 The two costs

There are two possible sources of regret.

First, exploration pulls the bad arm m times:

$$\text{exploration regret} = m\Delta.$$

Second, after exploration, the algorithm may wrongly commit to arm 2. If this happens, it pays roughly Δ for every remaining round:

$$\text{wrong-commit regret} \leq (T - 2m)\Delta \cdot \mathbb{P}(\hat{\mu}_2 \geq \hat{\mu}_1).$$

Thus

$$\mathbb{E}[\text{Reg}_T] \leq m\Delta + (T - 2m)\Delta \mathbb{P}(\hat{\mu}_2 \geq \hat{\mu}_1).$$

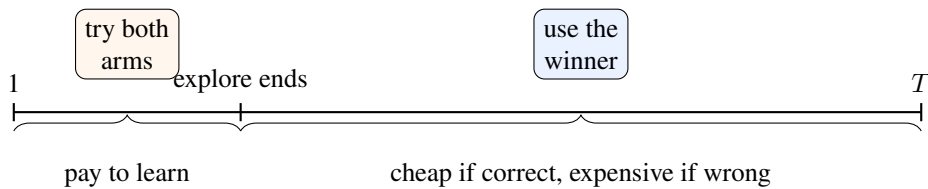


Figure 5.1: Explore-then-commit separates learning and earning. The separation makes the analysis easy and exposes what later algorithms improve.

5.3 Bounding the wrong commitment probability

The algorithm commits wrongly if

$$\hat{\mu}_2 \geq \hat{\mu}_1.$$

This event can happen only if at least one empirical mean makes a noticeable error.

Since

$$\mu_1 - \mu_2 = \Delta,$$

the midpoint between the two means is

$$\frac{\mu_1 + \mu_2}{2} = \mu_1 - \frac{\Delta}{2} = \mu_2 + \frac{\Delta}{2}.$$

If both estimates are on the correct side of this midpoint, then arm 1 wins:

$$\hat{\mu}_1 > \frac{\mu_1 + \mu_2}{2} > \hat{\mu}_2.$$

Therefore, the wrong event implies at least one failure:

$$\{\hat{\mu}_2 \geq \hat{\mu}_1\} \subseteq \left\{ \hat{\mu}_1 \leq \mu_1 - \frac{\Delta}{2} \right\} \cup \left\{ \hat{\mu}_2 \geq \mu_2 + \frac{\Delta}{2} \right\}.$$

For bounded rewards in $[0, 1]$, Hoeffding's inequality says

$$\mathbb{P}(\hat{\mu} - \mu \geq \varepsilon) \leq \exp(-2m\varepsilon^2),$$

and the same bound holds for downward deviations. With $\varepsilon = \Delta/2$,

$$\exp\left(-2m\left(\frac{\Delta}{2}\right)^2\right) = \exp\left(-\frac{m\Delta^2}{2}\right).$$

Using a union bound:

$$\mathbb{P}(E_1 \cup E_2) \leq \mathbb{P}(E_1) + \mathbb{P}(E_2).$$

So

$$\mathbb{P}(\hat{\mu}_2 \geq \hat{\mu}_1) \leq 2 \exp\left(-\frac{m\Delta^2}{2}\right).$$

Wrong commitment, line by line

$$\begin{aligned} \mathbb{P}(\hat{\mu}_2 \geq \hat{\mu}_1) &\leq \mathbb{P}\left(\hat{\mu}_1 \leq \mu_1 - \frac{\Delta}{2}\right) + \mathbb{P}\left(\hat{\mu}_2 \geq \mu_2 + \frac{\Delta}{2}\right) \\ &\leq \exp\left(-2m\left(\frac{\Delta}{2}\right)^2\right) + \exp\left(-2m\left(\frac{\Delta}{2}\right)^2\right) \\ &= 2 \exp\left(-\frac{m\Delta^2}{2}\right). \end{aligned}$$

Plugging this into regret gives

$$\mathbb{E}[\text{Reg}_T] \leq m\Delta + 2T\Delta \exp\left(-\frac{m\Delta^2}{2}\right).$$

5.4 What this simple algorithm teaches

Explore-then-commit is not the algorithm one should trust in a serious system. Its value is pedagogical. It shows the main bargain.

If m is small, the second term can be large: the learner may commit to the wrong arm. If m is large, the first term is large: the learner wastes too many trials on a known bad arm.

The bargain

Exploration is like paying rent for information. Too little rent and you stay ignorant. Too much rent and you never earn enough from what you learned.

The next idea, optimism, improves this by not fixing the exploration budget in advance.

Chapter 6 Optimism: Trust the Unknown Until It Disappoints You

6.1 The idea in one sentence

Optimism is not positivity. It is a rule for action under uncertainty.

Optimism

If an arm has not been tried much, allow it to look better than its current average. If it is truly bad, enough trials will shrink this allowance and remove it from serious competition.

This is the idea behind UCB: upper confidence bound.

At time t , for each arm a , compute

$$\text{index}_a(t) = \hat{\mu}_a(t) + r_a(t),$$

where $\hat{\mu}_a(t)$ is the observed average and $r_a(t)$ is a confidence radius. Choose the arm with the largest index.

$$A_t \in \operatorname{argmax}_a \{ \hat{\mu}_a(t) + r_a(t) \}.$$

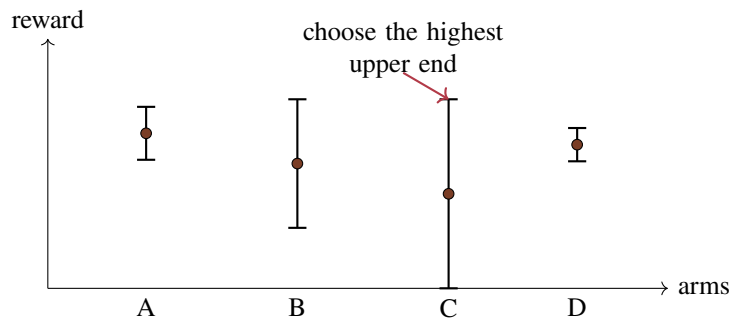


Figure 6.1: UCB chooses the arm with the best plausible value, not necessarily the best current average.

6.2 Where the radius comes from

Let an arm be pulled n times. Let its rewards be $Y_1, \dots, Y_n$, each in $[0, 1]$, with mean μ . The empirical mean is

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n Y_i.$$

Hoeffding's inequality says

$$\mathbb{P}(\hat{\mu}_n - \mu \geq r) \leq \exp(-2nr^2).$$

We want the right side to be a tiny number, say t^{-4} . So choose r by solving

$$\exp(-2nr^2) = t^{-4}.$$

Now solve it slowly:

Solving for the UCB radius

$$\begin{aligned}
\exp(-2nr^2) &= t^{-4} \\
\log(\exp(-2nr^2)) &= \log(t^{-4}) \\
-2nr^2 &= -4 \log t \\
2nr^2 &= 4 \log t \\
r^2 &= \frac{2 \log t}{n} \\
r &= \sqrt{\frac{2 \log t}{n}}.
\end{aligned}$$

Thus a natural confidence radius is

$$r_a(t) = \sqrt{\frac{2 \log t}{N_a(t)}}.$$

It shrinks when the arm is sampled often. It grows slowly with time because the algorithm makes many decisions and we want confidence to hold across many of them.

6.3 The good event

Here is the event we want:

$$|\hat{\mu}_a(t) - \mu_a| \leq r_a(t) \quad \text{for all arms and relevant times.}$$

Call this event G . On G , every confidence interval contains its true mean.

Why can we hope G holds? Because each individual failure is rare, and a union bound lets us add many rare failures.

Union bound in the UCB setting

For one arm and one sample size n ,

$$\mathbb{P}\left(\hat{\mu}_{a,n} - \mu_a > \sqrt{\frac{2 \log T}{n}}\right) \leq \exp\left(-2n \cdot \frac{2 \log T}{n}\right) = T^{-4}.$$

The lower tail is the same:

$$\mathbb{P}\left(\mu_a - \hat{\mu}_{a,n} > \sqrt{\frac{2 \log T}{n}}\right) \leq T^{-4}.$$

Therefore

$$\mathbb{P}\left(|\hat{\mu}_{a,n} - \mu_a| > \sqrt{\frac{2 \log T}{n}}\right) \leq 2T^{-4}.$$

There are at most KT pairs (a, n) , so

$$\mathbb{P}(G^c) \leq \sum_{a=1}^K \sum_{n=1}^T 2T^{-4} = 2KT^{-3}.$$

No mystery is hidden here. The good event is just the statement that all sample averages are close to their true means. The union bound is just the rule that if many bad things can happen, the chance that at least one happens is at most the sum of their chances.

6.4 Why a bad arm eventually loses

Let a be a suboptimal arm. Its gap is

$$\Delta_a = \mu_* - \mu_a > 0.$$

Suppose the good event G holds and UCB chooses arm a at time t . Since UCB chose a , its index must be at least the index of the best arm $*$:

$$\hat{\mu}_a(t) + r_a(t) \geq \hat{\mu}_*(t) + r_*(t).$$

On G ,

$$\hat{\mu}_*(t) + r_*(t) \geq \mu_*.$$

Also on G ,

$$\hat{\mu}_a(t) \leq \mu_a + r_a(t).$$

Thus

$$\hat{\mu}_a(t) + r_a(t) \leq \mu_a + 2r_a(t).$$

Putting the inequalities together:

$$\mu_* \leq \mu_a + 2r_a(t).$$

Therefore

$$\Delta_a \leq 2r_a(t).$$

Using $r_a(t) = \sqrt{2 \log T / N_a(t)}$:

$$\Delta_a \leq 2\sqrt{\frac{2 \log T}{N_a(t)}}.$$

Now solve for $N_a(t)$:

A bad arm can be chosen only while its radius is large

$$\begin{aligned} \Delta_a &\leq 2\sqrt{\frac{2 \log T}{N_a(t)}} \\ \frac{\Delta_a}{2} &\leq \sqrt{\frac{2 \log T}{N_a(t)}} \\ \frac{\Delta_a^2}{4} &\leq \frac{2 \log T}{N_a(t)} \\ N_a(t) &\leq \frac{8 \log T}{\Delta_a^2}. \end{aligned}$$

This is the entire UCB proof in one picture: a bad arm is played only while it can still hide behind uncertainty. Once its confidence interval is narrow enough, it cannot beat the best arm's lower reality anymore.

6.5 The regret bound

On the good event, each bad arm a is pulled at most about

$$\frac{8 \log T}{\Delta_a^2}$$

times after initialization. Its regret contribution is gap times count:

$$\Delta_a \cdot \frac{8 \log T}{\Delta_a^2} = \frac{8 \log T}{\Delta_a}.$$

Adding arms gives

$$\mathbb{E}[\text{Reg}_T] \lesssim \sum_{a: \Delta_a > 0} \frac{\log T}{\Delta_a}.$$

Takeaway

UCB is not a complicated formula. It is a disciplined way to say: try uncertain things while they could plausibly be good; stop trying them once the data have made that hope too expensive to believe.

Chapter 7 Sampling a Possible World

7.1 The Bayesian picture

Optimism says: choose the arm with the best plausible upper value.

Posterior sampling says something lighter:

Posterior sampling

Imagine one possible world consistent with the data. Act optimally in that imagined world. Repeat this every round.

This is Thompson sampling. It is one of the cleanest ideas in sequential learning. It often looks almost too simple: keep a posterior distribution over the unknown rewards, sample from it, and choose the arm that is best under the sample.

7.2 A coin-click model

Suppose each thumbnail has an unknown click probability θ_a . A click is 1, no click is 0:

$$X_t(a) \sim \text{Ber}(\theta_a).$$

For each arm, start with a Beta prior:

$$\theta_a \sim \text{Beta}(\alpha_a, \beta_a).$$

If the arm receives a click, increase α_a by one. If it receives no click, increase β_a by one.

Beta-Bernoulli update

$$\text{prior density: } p(\theta) \propto \theta^{\alpha-1}(1-\theta)^{\beta-1},$$

$$\text{one click: } p(1 | \theta) = \theta,$$

$$\begin{aligned} \text{posterior: } p(\theta | 1) &\propto \theta \cdot \theta^{\alpha-1}(1-\theta)^{\beta-1} \\ &= \theta^{\alpha}(1-\theta)^{\beta-1}, \\ \theta | 1 &\sim \text{Beta}(\alpha + 1, \beta). \end{aligned}$$

$$\text{one no-click: } p(0 | \theta) = 1 - \theta,$$

$$\begin{aligned} p(\theta | 0) &\propto (1-\theta)\theta^{\alpha-1}(1-\theta)^{\beta-1} \\ &= \theta^{\alpha-1}(1-\theta)^{\beta}, \\ \theta | 0 &\sim \text{Beta}(\alpha, \beta + 1). \end{aligned}$$

After S_a clicks and F_a no-clicks on arm a ,

$$\theta_a | \text{data} \sim \text{Beta}(\alpha_a + S_a, \beta_a + F_a).$$

7.3 The decision rule

At time t :

$$\tilde{\theta}_a(t) \sim \text{Beta}(\alpha_a + S_a(t), \beta_a + F_a(t)),$$

independently for each arm, and choose

$$A_t \in \operatorname{argmax}_a \tilde{\theta}_a(t).$$

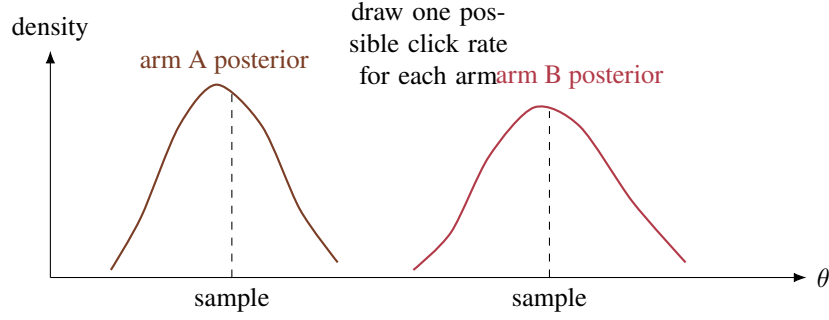


Figure 7.1: Thompson sampling does not directly add a confidence bonus. It samples a plausible world and follows the best arm in that world.

7.4 Why this explores

If an arm has little data, its posterior is wide. A wide posterior occasionally samples a high value. That gives the arm chances to be tried. If the arm is bad, the failures push its posterior down and make such lucky samples rarer.

So the exploration is not manually scheduled. It emerges from uncertainty.

Classical paradigm: probability matching

Thompson sampling chooses an arm with probability equal to the posterior probability that the arm is optimal. Arms are tried according to how believable their optimality is under current evidence.

Let A^* be the optimal arm under the unknown true parameter. Given the history $\mathcal{F}_t$, posterior sampling satisfies

$$\mathbb{P}(A_t = a \mid \mathcal{F}_t) = \mathbb{P}(A^* = a \mid \mathcal{F}_t).$$

This identity is the clean algebraic soul of Thompson sampling.

7.5 The identity slowly

Let θ be the unknown vector of arm means. Let

$$g(\theta) = \operatorname{argmax}_a \theta_a.$$

The true optimal arm is

$$A^* = g(\theta).$$

Thompson sampling draws

$$\tilde{\theta} \sim \text{posterior of } \theta \text{ given } \mathcal{F}_t,$$

and plays

$$A_t = g(\tilde{\theta}).$$

Because θ and $\tilde{\theta}$ have the same conditional distribution given $\mathcal{F}_t$,

$$\theta \mid \mathcal{F}_t \stackrel{d}{=} \tilde{\theta} \mid \mathcal{F}_t.$$

Therefore applying the same map g preserves the conditional distribution:

$$g(\theta) \mid \mathcal{F}_t \stackrel{d}{=} g(\tilde{\theta}) \mid \mathcal{F}_t.$$

Thus

$$A^* \mid \mathcal{F}_t \stackrel{d}{=} A_t \mid \mathcal{F}_t.$$

So for each arm a ,

$$\mathbb{P}(A^* = a \mid \mathcal{F}_t) = \mathbb{P}(A_t = a \mid \mathcal{F}_t).$$

This proof uses no heavy probability. It says: if two hidden worlds are drawn from the same bag, then the winner in the first world and the winner in the second world have the same chance of being any given arm.

7.6 Information accounting

Many modern Thompson sampling analyses, including work in the broader line of information-ratio proofs, ask a simple question:

How much regret do we pay per unit of information gained?

The technical details can become intricate, but the idea is almost everyday reasoning. If a trial is expensive but teaches a lot, it may be worth it. If a trial is expensive and teaches little, a good algorithm should avoid repeating it.

This way of thinking connects cleanly to recent theoretical work on Thompson sampling, best-arm identification with regret control, and multi-agent bandits, including work by Jin and collaborators [2, 3, 7]. The common flavor is not just to design a rule, but to prove that the rule pays for uncertainty in a controlled way.

Chapter 8 Why This Toy Model Keeps Reappearing

8.1 Context changes the arms

The homepage example was too simple: every visitor was treated the same. In practice, users arrive with context: time, device, language, past behavior, location, or topic preference.

A contextual bandit sees a context x_t , chooses an action A_t , and receives the reward of that action. The missing counterfactual remains:

$$\text{observe } X_t(A_t, x_t), \quad \text{miss } X_t(a, x_t), \quad a \neq A_t.$$

The setting becomes richer, but the wound is the same.

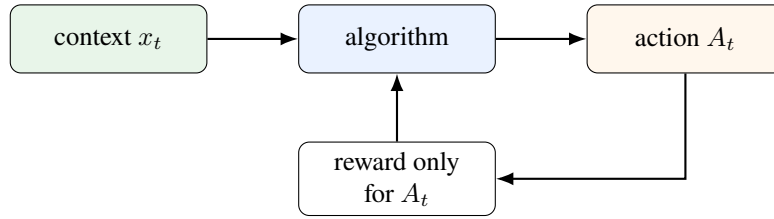


Figure 8.1: Contextual bandits add side information before the action. They do not restore the missing outcomes of unchosen actions.

8.2 Linear bandits: the geometry version

In a linear bandit, each action has a feature vector $x_a \in \mathbb{R}^d$, and the mean reward is assumed to be approximately linear:

$$\mu_a = x_a^\top \theta_*.$$

The unknown object is now a vector θ_* . Each action gives one noisy measurement of this vector in the direction x_a .

A helpful picture is this: learning a linear bandit is like trying to locate a hidden point in space by shining flashlights from different directions. Some directions reveal new information. Others repeat what is already known.

The UCB idea becomes geometric:

$$\text{score}(x) = x^\top \hat{\theta} + \beta \sqrt{x^\top V^{-1} x}.$$

The first term is what we currently believe. The second term is how uncertain we are in direction x .

Linear confidence in one line

$$\begin{aligned}
 |x^\top (\hat{\theta} - \theta_*)| &= |\langle x, \hat{\theta} - \theta_* \rangle| \\
 &= |\langle V^{-1/2} x, V^{1/2} (\hat{\theta} - \theta_*) \rangle| \\
 &\leq \|V^{-1/2} x\|_2 \cdot \|V^{1/2} (\hat{\theta} - \theta_*)\|_2 \\
 &\leq \sqrt{x^\top V^{-1} x} \cdot \beta.
 \end{aligned}$$

The last inequality is just Cauchy-Schwarz plus a confidence ellipsoid. This is the geometric heart of linear UCB.

8.3 Gaussian-process bandits: smooth unknown landscapes

Sometimes actions are not a small list. They may be continuous: temperature in a lab, a hyperparameter, a drug dosage, a robotics controller, or a design choice. The reward function may be smooth but unknown.

Gaussian-process bandits model the unknown function as a random smooth landscape. The algorithm samples or optimizes where the landscape might be high, while learning the shape of the landscape from noisy evaluations.

The same old idea appears again:

mean prediction + uncertainty bonus.

For GP-UCB this takes the form

$$A_t \in \operatorname{argmax}_{x \in \mathcal{X}} \left\{ m_{t-1}(x) + \sqrt{\beta_t \sigma_{t-1}(x)} \right\}.$$

This line connects finite-armed bandits to Bayesian optimization, a major line of work including Srinivas, Krause, Kakade, and Seeger [6], the Cambridge probabilistic modelling tradition [1], and Oxford work on Gaussian-process global optimization [5].

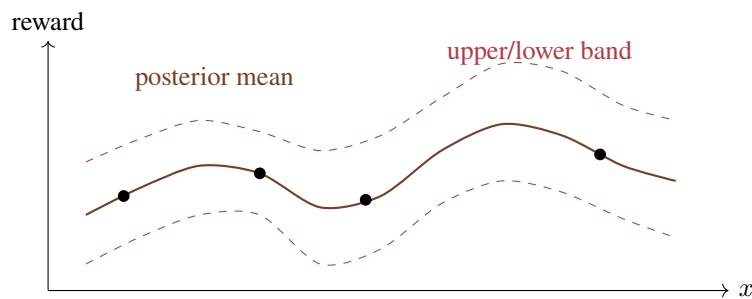


Figure 8.2: In GP bandits, the confidence interval becomes a band over a function. The old finite-arm picture survives, but the arms now live in a space.

8.4 Reliable AI and the same old missing column

The same obstruction appears in modern systems. A recommender only observes the item it showed. A tutoring system only observes the hint it gave. A medical decision support tool may only observe the outcome under the chosen treatment. An LLM steering policy only observes the response produced under the chosen prompt or intervention.

The names change. The missing counterfactual remains.

This is why bandits are not just a topic. They are a microscope. They strip sequential decision-making down to its smallest difficult piece.

Epilogue: The Essence

The whole note can be compressed into a few sentences.

Supervised learning receives examples. Bandit learning creates examples by acting. Because only the chosen action is observed, the learner must spend some reward to buy information. Regret is the receipt for that purchase.

Explore-then-commit buys information on a fixed schedule. UCB buys it adaptively by giving uncertain arms temporary credit. Thompson sampling buys it by sampling possible worlds and acting according to their winners. Linear and Gaussian-process bandits carry the same ideas into structured spaces.

The deepest lesson is not a formula. It is a research habit: before designing an algorithm, ask what feedback is actually visible. Most of the mathematics follows from that one question.

References

- [1] Z. Ghahramani. “Probabilistic machine learning and artificial intelligence”. In: *Nature* 521 (2015), pp. 452–459.
- [2] T. Jin et al. “Finite-time frequentist regret bounds of multi-agent Thompson sampling on sparse hypergraphs”. In: *AAAI Conference on Artificial Intelligence*. 2024.
- [3] T. Jin et al. *Finite-time regret of Thompson sampling algorithms for exponential family multi-armed bandits*. arXiv:2206.03520. 2022.
- [4] T. Lattimore and C. Szepesvari. *Bandit Algorithms*. Cambridge University Press, 2020.
- [5] M. A. Osborne, R. Garnett, and S. J. Roberts. “Gaussian processes for global optimization”. In: *International Conference on Learning and Intelligent Optimization*. 2009.
- [6] N. Srinivas et al. “Gaussian process optimization in the bandit setting: No regret and experimental design”. In: *International Conference on Machine Learning*. 2010.
- [7] J. Yang, V. Y. F. Tan, and T. Jin. *Best arm identification with minimal regret*. arXiv:2409.18909. 2024.